Instructions RISC-V – RV32I

Instructions de transferts

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Opérations de transferts ALU (add/load upper immediate [pc])					
`auipc`	d, C	Chargement immédiat	Calcul avec PC	$d \leftarrow \mathrm{PC} + (C \ll 12)$	U
`lui`	d, C	Chargement immédiat	Chargement d'une constante	$d \leftarrow C \ll 12$	U
Opérations de transferts registre-RAM (load/store byte/half/word...)					
`lb`	d, C(s)	Chargement octet	Extension de signe	$d \leftarrow \mathrm{RAM}[s+C]$	I
`lbu`	d, C(s)	Chargement octet non-signé	Sans extension de signe	$d \leftarrow \mathrm{RAM}[s+C]$	I
`lh`	d, C(s)	Chargement demi-mot	Extension de signe	$d \leftarrow \mathrm{RAM}[s+C]$	I
`lhu`	d, C(s)	Chargement demi-mot non-signé	Sans extension de signe	$d \leftarrow \mathrm{RAM}[s+C]$	I
`lw`	d, C(s)	Chargement mot		$d \leftarrow \mathrm{RAM}[s+C]$	I
`sb`	s, C(d)	Rangement octet		$\mathrm{RAM}[d+C] \leftarrow s$	S
`sh`	s, C(d)	Rangement demi-mot		$\mathrm{RAM}[d+C] \leftarrow s$	S
`sw`	s, C(d)	Rangement mot		$\mathrm{RAM}[d+C] \leftarrow s$	S

Instructions de branchement de code

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Saut conditionnel (branch if equal/not equal/less than/greater or equal...)					
`beq`	s1, s2, *Etq*	Branchement si égalité		$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 = s2$	B
`bne`	s1, s2, *Etq*	Branchement si différent		$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 \neq s2$	B
`blt`	s1, s2, *Etq*	Branchement si s1 inf. à s2		$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 < s2$	B
`bge`	s1, s2, *Etq*	Branchement si s1 sup. ou égal s2		$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 \geq s2$	B
`bltu`	s1, s2, *Etq*	Branchement si s1 inf. à s2	Non-signé	$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 < s2$	B
`bgeu`	s1, s2, *Etq*	Branchement si s1 sup. ou égal s2	Non-signé	$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 \geq s2$	B
Saut inconditionnel (jump and link [register])					
`jal`	d, *Etq*	Saut et liaison	$d \leftarrow$ adr. retour	$d \leftarrow \mathrm{PC} + 4$; $\mathrm{PC} \leftarrow \mathrm{PC} + Etq$	J
`jalr`	d, C(s)	Saut et liaison	$d \leftarrow$ adr. retour	$d \leftarrow \mathrm{PC} + 4$; $\mathrm{PC} \leftarrow s + C$	I

Instructions arithmétiques

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Opérations arithmétiques (uniquement addition/soustraction)					
`add`	d, s1, s2	addition	Sans détection de dépassement	$d \leftarrow s1 + s2$	R
`sub`	d, s1, s2	soustraction	Sans détection de dépassement	$d \leftarrow s1 - s2$	R
`addi`	d, s, C	addition (op. imm.)	Sans détection de dépassement	$d \leftarrow s + C$	I

Instructions logiques

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Opérations logiques de base (bits-à-bits)					
`or`	d, s1, s2	OU logique		$d \leftarrow s1 \mid s2$	R
`and`	d, s1, s2	ET logique		$d \leftarrow s1 \,\&\, s2$	R
`xor`	d, s1, s2	OU exclusif logique		$d \leftarrow s1 \hat{} s2$	R
`ori`	d, s, C	OU logique avec op. imm.	Immédiat non-signé	$d \leftarrow s \mid C$	I
`andi`	d, s, C	ET logique avec op. imm.	Immédiat non-signé	$d \leftarrow s \,\&\, C$	I
`xori`	d, s, C	OU exclusif avec op. imm.	Immédiat non-signé	$d \leftarrow s \hat{} C$	I
Opérations de décalage de bits (shift left/right logical)					
`sll`	d, s1, s2	Décalage à gauche		$d \leftarrow s1 \ll s2[4{:}0]$	R
`srl`	d, s1, s2	Décalage à droite		$d \leftarrow s1 \gg s2[4{:}0]$	R
`sra`	d, s1, s2	Décalage à droite arithmétique	Extension de signe	$d \leftarrow s1 \gg^{*} s2[4{:}0]$	R
`slli`	d, s, C	Décalage à gauche		$d \leftarrow s \ll C$	I
`srli`	d, s, C	Décalage à droite		$d \leftarrow s \gg C$	I
`srai`	d, s, C	Décalage à droite arithmétique	Extension de signe	$d \leftarrow s \gg^{*} C$	I
Opérations de test de conditions (set if less than)					
`slt`	d, s1, s2	positionné si inf. à		$d \leftarrow 1$ si $(s1 < s2)$ $d \leftarrow 0$ sinon	R
`sltu`	d, s1, s2	positionné si inf. à (op. non-signé)		$d \leftarrow 1$ si $(s1 < s2)$ $d \leftarrow 0$ sinon	R
`slti`	d, s, C	positionné si inf. à (op. immédiat)	Extension de signe pour l'immédiat	$d \leftarrow 1$ si $(s < C)$ $d \leftarrow 0$ sinon	I
`sltiu`	d, s, C	positionné si inf. à (op. immédiat, non-signé)		$d \leftarrow 1$ si $(s < C)$ $d \leftarrow 0$ sinon	I

Système et debuguer

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Appels divers – (environment call & debuger)					
`ebreak`		appel débogueur		Transfert du contrôle au débogueur	I
`ecall`		appel système		Transfert du contrôle au SE	I

Instructions non-étudiées dans le module

- Deux instructions `fence` et `fence.i` pour la synchronisation (mécanisme de barrière).
- Six instructions `CSRR{W, S, C, WI, SI, CI}` pour manipuler les registres de contrôle et de statut.

Pseudo-instructions

Rappel : les pseudo-instructions constituent des facilités d'écritures/lecture des programmes assembleurs. Les pseudo-instructions suivantes sont celles faisant partie du standard. Le logiciel RARS est susceptible d'en proposer davantage. Ces pseudo-instructions sont très pratiques notamment pour les branchements.

Pseudo-instructions de transfert et construction d'entier

Syntaxe Assembleur		*Traduction*	Signification
Transferts et construction d'entier			
`la`	d, *Etq*	`auipc d, Etq[31:12]` `addi d, d, Etq[11:0]`	chargement d'une adresse
`li`	d, C	suite d'instructions avec immédiat	chargement d'une constante
`l{b,h,w}`	d, *Etq*, s	`auipc s, Etq[31:12]` `l{b,h,w} d, Etq[11:0](s)`	chargement depuis une adresse
`mv`	d, s	`addi d, s, 0`	copie de s vers d
`s{b,h,w}`	d, *Etq*, s	`auipc s, Etq[31:12]` `s{b,h,w}, Etq[11:0](d)`	rangement vers une adresse

Pseudo-instructions de test

Syntaxe Assembleur		*Traduction*	Signification
Pseudo-instructions de test (variations)			
`seqz`	d, s	`sltiu d, s, 1`	1 si s vaut zéro
`snez`	d, s	`sltu d, zero, s`	1 si s diff. de zéro
`sltz`	d, s	`slt d, s, zero`	1 si s inf. à zéro
`sgtz`	d, s	`slt d, zero, s`	1 si s sup. à zéro

Pseudo-instructions pour saut et appel de fonction

Syntaxe Assembleur		*Traduction*	Signification
Saut/fonction			
`j`	*Etq*	`jal zero, Etq`	Saut sur *Etq*
`jal`	*Etq*	`jal ra, Etq`	Saut sur *Etq*avec liaison
`jr`	s	`jalr zero, 0(s)`	Saut sur registre
`jalr`	s	`jalr ra, 0(s)`	Saut et liaison sur registre
`ret`		`jalr zero, 0(ra)`	« return »
`call`	*Etq*	`auipc t1, Etq[31:12]` `jalr ra, t1, Etq[11:0]`	Saut sur *Etq*éloignée avec liaison
`tail`	*Etq*	`auipc t1, Etq[31:12]` `jalr zero, t1, Etq[11:0]`	Fin fonction site éloigné

Pseudo-instructions de branchement

Syntaxe Assembleur		*Traduction*	Signification
Branchements (variations)			
`beqz`	s, *Etq*	`beq s, zero, Etq`	Branchement sur *Etq*si $s = 0$
`bnez`	s, *Etq*	`bne s, zero, Etq`	Branchement sur *Etq*si $s \neq 0$
`blez`	s, *Etq*	`bge zero, s, Etq`	Branchement sur *Etq*si $s \leq 0$
`bgez`	s, *Etq*	`bge s, zero, Etq`	Branchement sur *Etq*si $s \geq 0$
`bltz`	s, *Etq*	`blt s, zero, Etq`	Branchement sur *Etq*si $s < 0$
`bgtz`	s, *Etq*	`blt zero, s, Etq`	Branchement sur *Etq*si $s > 0$
`bgt`	s1, s2, *Etq*	`blt s2, s1, Etq`	Branchement sur *Etq*si $s1 > s2$
`ble`	s1, s2, *Etq*	`bge s2, s1, Etq`	Branchement sur *Etq*si $s1 \leq s2$
`bgtu`	s1, s2, *Etq*	`bltu s2, s1, Etq`	Branchement sur *Etq*si $s1 > s2$ (non-signé)
`bleu`	s1, s2, *Etq*	`bgeu s2, s1, Etq`	Branchement sur *Etq*si $s1 \leq s2$ (non-signé)

Divers

Syntaxe Assembleur		*Traduction*	Signification
Divers (mémoire, calcul, pipeline)			
`fence`		`fence iorw, iorw`	Barrière sur mémoires et E/S
`neg`	d, s	`sub d, zero, s`	Complément à deux
`nop`		`addi zero, zero, 0`	*No operation*
`not`	d, s	`xori d, s, -1`	Complément à un

Le lecteur trouvera de plus amples détails au niveau des différentes références proposées ici :

`https://riscv.org`

Directives assembleur

Indiquer le segment mémoire

- `.text [adr]` : la donnée sera dans le segment texte (*i.e.* avec les instructions du programme – à l'adresse `adr` si fournie).
- `.data [adr]` : la donnée sera stockée dans le segment des données (à l'adresse `adr` si fournie).
- `.stack` : la donnée sera sur la pile.
- `.k{text,data}` : la donnée sera dans le segment {texte, données} du noyau du système d'exploitation.

Décrire la visibilité

- `.extern` *`Etq`* `taille` : déclare la donnée enregistrée à l'adresse *Etq* sur `taille` octets consécutifs comme étant globale au fichier. Place la donnée dans le segment des données.
- `.globl` *`Etq`* : déclare l'étiquette *Etq* est un symbole globale visible dans d'autres fichiers assembleur.

Décrire et enregistrer les données Les enregistrements mémoire suivants sont tous réalisés à des adresses consécutives.

- `.align n` : réalise l'alignement mémoire de la donnée suivante sur une frontière de 2^n octets.
- `.space n` : débute un espace de n octets consécutifs dans le segment des données.
- `.ascii ch` : enregistre en mémoire la chaîne de caractère `ch` sans le caractère de fin de chaîne.
- `.asciz ch` : idem, mais avec le caractère de fin de chaîne.
- `.byte` $b_1, \ldots, b_n$: enregistre n octets.
- `.double` $d_1, \ldots, d_n$: enregistre n flottants double précision.
- `.float` $f_1, \ldots, f_n$: enregistre n flottants simple précision.
- `.half` $h_1, \ldots, h_n$: enregistre n quantités 16 bits (alignement demi-mot).
- `.word` $w_1, \ldots, w_n$: enregistre n quantités 32 bits (alignement mot).

Appels système

Méthodologie

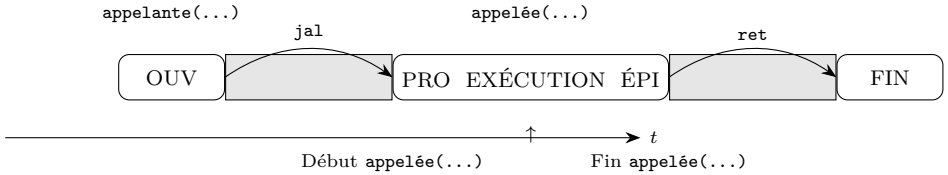
1. Écrire l'instruction pour charger le service voulu dans le registre `a7`.
RISC-V).
2. Écrire l'instruction `ecall`.
l `ecall`.

Service	Code Service	Arguments	Valeur de retour
afficher entier	1	`a0` : stocke l'entier voulu	
afficher flottant	2	`fa0` : stocke le flottant voulu	
afficher ch. car.	4	`a0` : adresse de la chaîne	
afficher caractère	11	`a0` : stocke le caractère voulu	
lire entier	5		`a0` : entier lu
lire flottant	6		`fa0` : entier lu
lire ch. car.	8	`a0` : adresse du tampon `a1` : nb max car. lu	
lire ch. car.	8	`a0` : adresse du tampon	
lire caractère	12		`a0` : caractère lu
`exit`	10		

L'onglet **Syscalls** de la section d'aide de RARS présente d'autres appels système pour la génération de nombres aléatoires, la manipulation de fichier…

Fonction

Méthodologie Procéder en 5 étapes : ouverture, prologue, exécution, épilogue, final.



1. **OUV**erture : l'appelante prépare la réalisation effective de l'appelée (sauvegarde éventuelle de registres).
2. **PRO**logue : l'appelée prépare son bloc de données sur la pile.
3. **EXECUTION** : l'appelée exécute ses instructions propres.
4. **EPI**logue : l'appelée rend la main en restituant le contexte de l'appelant et en dépilant son bloc mémoire de la pile.
5. **FIN**al : l'appelante restaure éventuellement des registres et reprend le code de la fonction implémentée.

Registres à sauvegarder lors d'un appel

Nom ABI/RARS	Numéro (précédé de x)	Usage (convention)	Appelante	Appelée
`zero`	0	constante 0	–	–
`ra`	1	Adr. retour	[✓]	–
`sp`	2	Pointeur pile	–	[✓]
`gp`	3	Pointeur global	–	–
`tp`	4	Pointeur *thread*	–	–
`t0`	5	Temp./liaison fonct.	[✓]	–
`t1 - t2`	6 – 7	Temp.	[✓]	–
`s0/fp`	8	Sauv./déb. bloc pile	–	[✓]
`s1`	9	Sauv.	–	[✓]
`a0 - a1`	10 – 11	Arguments/Résultat	[✓]	–
`a2 - a7`	12 – 17	Arguments	[✓]	–
`s2 - s11`	18 – 27	Sauv.	–	[✓]
`t3 - t6`	28 – 31	Temp.	[✓]	–
`pc`		compteur ordinal		

Toutes les sauvegardes de registres sont codifiées par la convention décrite précédemment. Cependant, la sauvegarde d'un registre donné n'est pas nécessairement systématique. Ce caractère non-systématique est traduit dans le tableau ci-dessus par une notation entre crochets (usuellement utilisée dans la syntaxe des options en programmation). Une entrée du tableau entre crochets se trouve conditionnée d'une part par le fait qu'un appel de fonction a bien eu lieu, et d'autre part selon l'usage ou non qui est fait du registre par l'appelante ou l'appelée. Par exemple, si la fonction appelante a besoin d'accéder aux valeurs qu'elle a stockées dans les registres `t0`–`t6`, elle doit les sauvegarder dans son bloc de pile avant l'appel de fonction car ces registres sont susceptibles d'être modifiés par la fonction appelée sans précaution particulière. Si ce n'est pas le cas, la fonction appelante ne les empile pas.

Encodage des données

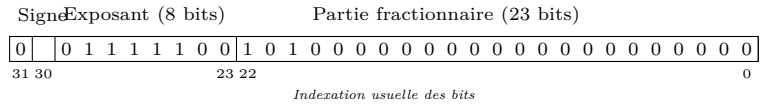
Table d'encodage ASCII

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	␣	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Exemple : le caractère `'A'` est encodé par 65 ($4 \times 16 + 1$), ou encore $(41)_{16}$.

Remarque : cette table est aussi facilement accessible sur les systèmes d'exploitation de type GNU/Linux en tapant `man ascii` dans un terminal.

Format d'encodage des flottants simple précision IEEE-754



Remarques :

- L'exposant encodé est un exposant biaisé. Il faut retrancher le biais $B = 127$ de cette valeur pour obtenir l'exposant réel du nombre encodé.
- Seule la partie fractionnaire de la notation scientifique normalisée est encodée dans la mantisse. Ainsi, l'exemple ci-dessus encode le nombre 0,15625 (*cf.* CM1 pour plus de détails sur ce calcul).
- Pour un flottant avec double précision, les champs exposant et partie fractionnaire occupent respectivement 11 et 52 bits. Le biais vaut 1023.

Format d'encodage des entiers en complément à deux

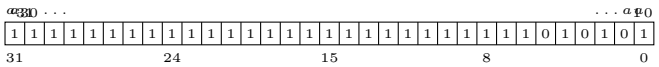
Sur la plupart des ordinateurs, les entiers sont encodés en complément à deux.

$$(a_{n-1}a_{n-2}\dots a_1a_0)_{cp2} = -a_{n-1}\cdot 2^{n-1} + a_{n-2}\cdot 2^{n-2} + \dots + a_1\cdot 2^1 + a_0\cdot 2^0$$

Technique d'encodage :

1. encoder le module de l'entier en binaire ;
2. si l'entier est négatif, complémenter tous les bits et ajouter un.

Exemple sur un mot de 32 bits (avec ici l'entier -43) :



Quelques extensions

RV32M – *Multiplication extension* (opcode 011 0011 – fonct7 000 0001)

Syntaxe Assembleur		Opérations	fonct3	Effet	Format
		Division			
`div`	d, s1, s2	division	100	$d \leftarrow s1/s2$	R
`divu`	d, s1, s2	idem non-signé	101	$d \leftarrow s1/s2$	R
`rem`	d, s1, s2	reste de division	110	$d \leftarrow \text{reste}(s1/s2)$	R
`remu`	d, s1, s2	idem, non-signé	111	$d \leftarrow \text{reste}(s1/s2)$	R
		Multiplication			
`mul`	d, s1, s2	partie basse	000	$d \leftarrow (s1 \times s2)[31{:}0]$	R
`mulh`	d, s1, s2	partie haute	001	$d \leftarrow (s1 \times s2)[63{:}32]$	R
`mulhsu`	d, s1, s2	idem	010	$d \leftarrow (s1 \times s2)[63{:}32]$	R
`mulhu`	d, s1, s2	idem non-signée	011	$d \leftarrow (s1 \times s2)[63{:}32]$	R

RV32A – *Atomic Extension* (opcode 011 0011 – fonct3 010)

31 . . . 27	26 25	24 . . . 20	19 . . . 15	14 . . . 12	11 . . . 7	6 . . . 0
fonct5	aq rl	s2	s1	fonct3	d	opcode

Syntaxe Assembleur		Opérations	fonct5	Effet	Format
		Chargement/Rangement			
`lr.w`	d, s1, s2	Chargement réservé	00010	$d \leftarrow \text{RAM}[s1]$ réserve $\text{RAM}[s1]$	R
`sc.w`	d, s1, s2	Rangement conditionnel	00011	Si (réservé) $\text{RAM}[s1] \leftarrow s2$ $d \leftarrow 0$ Sinon $d \leftarrow 1$	R
		Opérations arithmétiques & logiques			
`amoadd.w`	d, s1, s2	Addition atomique	00000	$d \leftarrow \text{RAM}[s1] + s2$ $\text{RAM}[s1] \leftarrow d$	R
`amoand.w`	d, s1, s2	Et atomique	01100	$d \leftarrow \text{RAM}[s1] \,\&\, s2$ $\text{RAM}[s1] \leftarrow d$	R
`amoor.w`	d, s1, s2	Ou atomique	01010	$d \leftarrow \text{RAM}[s1] \mid s2$ $\text{RAM}[s1] \leftarrow d$	R
`amoxor.w`	d, s1, s2	Ou exclusif atomique	00100	$d \leftarrow \text{RAM}[s1]\hat{}s2$ $\text{RAM}[s1] \leftarrow d$	R
		Opérations diverses			
`amomax.w`	d, s1, s2	Max atomique	10100	$d \leftarrow \max(\text{RAM}[s1], s2)$ $\text{RAM}[s1] \leftarrow d$	R
`amomin.w`	d, s1, s2	Min atomique	10000	$d \leftarrow \min(\text{RAM}[s1], s2)$ $\text{RAM}[s1] \leftarrow d$	R
`amoswap.w`	d, s1, s2	Échange atomique	00001	$d \leftarrow \text{RAM}[s1]$ $\text{swap}(d, s2)$ $\text{RAM}[s1] \leftarrow d$	R

Les flottants simple précision avec RISC-V (l'extension RV32F)

Cette extension permet de manipuler et calculer des nombres flottants simple précision. Pour cela, l'architecture dispose de 32 registres flottants analogues aux registres entiers au niveau de leur usage.

Nom RARS	Numéro (précédé de f)	Usage (convention)	Appelante	Appelée
`ft0 - ft7`	0 – 7	Temp.	[✓]	–
`fs0 - fs1`	8 – 9	Sauv.	–	[✓]
`fa0 - fa1`	10 – 11	Arguments/Résultats	[✓]	–
`fa2 - fa7`	12 – 17	Arguments	[✓]	–
`fs2 - fs11`	18 – 27	Sauv.	–	[✓]
`ft8 - ft11`	28 – 31	Temp.	[✓]	–

Sélection d'instructions flottantes

Voici une liste non-exhaustive des instructions de l'extension pour les flottants simple-précision. Certaines instructions spécifiques à l'écriture de programmes avec les flottants ont été omises ici (les opérations de « multiplication-addition », l'opération de classification, les opérations d'injection du signe, les opérations d'interprétation littérale de motif binaire).

Instructions arithmétiques

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Quatre opérations arithmétiques					
`fadd.s`	d, s1, s2	addition		$d \leftarrow s1 + s2$	R
`fsub.s`	d, s1, s2	soustraction		$d \leftarrow s1 - s2$	R
`fmul.s`	d, s1, s2	multiplication		$d \leftarrow s1 \times s2$	R
`fdiv.s`	d, s1, s2	division		$d \leftarrow s1/s2$	R

Instructions mathématiques

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
max, min & racine carrée					
`fmax.s`	d, s1, s2	maximum		$d \leftarrow \max\{s1, s2\}$	R
`fmin.s`	d, s1, s2	minimum		$d \leftarrow \min\{s1, s2\}$	R
`fsqrt.s`	d, s	racine carrée		$d \leftarrow \sqrt{s}$	R

Instructions de transferts

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Opérations de transferts registre-RAM (load/store word)					
`flw`	d, C(s)	Chargement flottant		$d \leftarrow \mathrm{RAM}[s + C]$	I
`fsw`	s, C(d)	Rangement flottant		$\mathrm{RAM}[d + C] \leftarrow s$	S

Instructions de comparaison

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Opérations de comparaisons de flottants (equal/less than/less equal)					
`feq.s`	d, s1, s2	positionné si égal à		$d \leftarrow 1$ si $(s1 = s2)$ $d \leftarrow 0$ sinon	R
`flt.s`	d, s1, s2	positionné si inf. à		$d \leftarrow 1$ si $(s1 < s2)$ $d \leftarrow 0$ sinon	R
`fle.s`	d, s1, s2	positionné si inf. ou égal à		$d \leftarrow 1$ si $(s1 \leq s2)$ $d \leftarrow 0$ sinon	R

Instructions de conversion

Syntaxe Assembleur		Opérations	Commentaires	Effet	Format
Opérations de conversion entre flottants simple précision et (mots) entiers					
`fcvt.s.w`	d, s1	conv. vers flottant		$d \leftarrow (\mathrm{float})\, s$	R
`fcvt.s.wu`	d, s1	conv. vers flottant (depuis entier non-signé)		$d \leftarrow (\mathrm{float})\, s$	R
`fcvt.w.s`	d, s1	conv. depuis flottant		$d \leftarrow (\mathrm{int})\, s$	R
`fcvt.wu.s`	d, s1	conv. depuis flottant (vers entier non-signé)		$d \leftarrow (\mathrm{uint})\, s$	R

Remarque : le champ `fonct3` du format R s'appelle `rm` pour les flottants. Il sert à définir le mode d'arrondi lors des calculs.

Pseudo-instructions flottantes

Pseudo-instructions de transfert

Syntaxe Assembleur		*Traduction*	Effet
Chargement/rangement global			
`flw`	d, *Etq*, s	`auipc s, Etq[31:12]` `flw d, Etq[11:0](s)`	chargement
`fsw`	d, *Etq*, s	`auipc s, Etq[31:12]` `fls s, Etq[11:0](d)`	rangement

Pseudo-instructions diverses

Syntaxe Assembleur		*Traduction*	Effet
Déplacement/val. abs./négation			
`fmv.s`	d, s	`fsgnj.s d, s, s`	déplacement
`fabs.s`	d, s	`fsgnjx.s d, s, s`	val. abs.
`fneg.s`	d, s	`fsgnjn.s d, s, s`	négation

Encodage des instructions – RV32I

Nom ABI/RARS	a0	a1	a2	a3	a4	a5	a6	a7	gp	ra	s0	s1	s2	s3	s4	s5
Numéro (préf. x)	10	11	12	13	14	15	16	17	3	1	8	9	18	19	20	21

Nom ABI/RARS	s6	s7	s8	s9	s10	s11	sp	t0	t1	t2	t3	t4	t5	t6	tp	zero
Numéro (préf. x)	22	23	24	25	26	27	2	5	6	7	28	29	30	31	4	0

Format R – Register

31 … 25	24 … 20	19 … 15	14 … 12	11 … 7	6 … 0
funct7	s2	s1	funct3	d	opcode

Syntaxe Assembleur		**Opcode**	**funct3**	**funct7**	**Effet**
Opérations arithmétiques (uniquement addition/soustraction)					
`add`	d, s1, s2	011 0011	000	000 0000	$d \leftarrow s1 + s2$
`sub`	d, s1, s2	011 0011	000	010 0000	$d \leftarrow s1 - s2$
Opérations logiques de base (bits-à-bits)					
`and`	d, s1, s2	011 0011	111	000 0000	$d \leftarrow s1 \,\&\, s2$
`or`	d, s1, s2	011 0011	110	000 0000	$d \leftarrow s1 \mid s2$
`xor`	d, s1, s2	011 0011	100	000 0000	$d \leftarrow s1 \hat{} s2$
Opérations de décalage de bits (shift left/right logical)					
`sll`	d, s1, s2	011 0011	001	000 0000	$d \leftarrow s1 \ll s2[4{:}0]$
`srl`	d, s1, s2	011 0011	101	000 0000	$d \leftarrow s1 \gg s2[4{:}0]$
`sra`	d, s1, s2	011 0011	101	010 0000	$d \leftarrow s1 \gg^{*} s2[4{:}0]$
Opérations de test de condition (set if less than)					
`slt`	d, s1, s2	011 0011	010	000 0000	$d \leftarrow 1$ si $(s1 < s2)$ $d \leftarrow 0$ sinon
`sltu`	d, s1, s2	011 0011	011	000 0000	$d \leftarrow 1$ si $(s1 < s2)$ $d \leftarrow 0$ sinon

Format S – Store

31 … 25	24 … 20	19 … 15	14 … 12	11 … 7	6 … 0
imm[11:5]	s	d	funct3	imm[4:0]	opcode

Syntaxe Assembleur		**Opcode**	**funct3**	**Immédiat : C**	**Effet**
Opérations de transferts registre vers RAM (store byte/half/word...)					
`sb`	s, C(d)	010 0011	000	bi-réparti	RAM$[d+C] \leftarrow s$
`sh`	s, C(d)	010 0011	001	bi-réparti	RAM$[d+C] \leftarrow s$
`sw`	s, C(d)	010 0011	010	bi-réparti	RAM$[d+C] \leftarrow s$

Format U – *Upper Immediate*

31 … 12	11 … 7	6 … 0
imm[31:12]	d	opcode

Syntaxe Assembleur		**Opcode**	**Immédiat : C**	**Effet**
Saut inconditionnel (jump and link)				
`auipc`	d, C	001 0111		$d \leftarrow \mathrm{PC} + (C \ll 12)$
`lui`	d, C	011 0111		$d \leftarrow C \ll 12$

Format J – Jump

31 … 12	11 … 7	6 … 0
imm[20 | 10:1 | 11 | 19:12]	d	opcode

Syntaxe Assembleur		**Opcode**	**Immédiat : *Etq***	**Effet**
Saut inconditionnel (jump and link)				
`jal`	d, *Etq*	110 1111	Scindé et permuté	$d \leftarrow \mathrm{PC} + 4$ $\mathrm{PC} \leftarrow \mathrm{PC} + Etq$

Format I – *Immediate*

31 … 20	19 … 15	14 … 12	11 … 7	6 … 0
imm[11:0]	s	funct3	d	opcode

Syntaxe Assembleur		**Opcode**	**funct3**	**Immédiat : C ou *Etq***	**Effet**
Opérations arithmétiques (uniquement addition)					
`addi`	d, s, C	001 0011	000	C codé sur 12 bits	$d \leftarrow s + C$
Opérations logiques de base (bits-à-bits)					
`andi`	d, s, C	001 0011	111	C codé sur 12 bits	$d \leftarrow s \,\&\, C$
`ori`	d, s, C	001 0011	110	C codé sur 12 bits	$d \leftarrow s \mid C$
`xori`	d, s, C	001 0011	100	C codé sur 12 bits	$d \leftarrow s \hat{} C$
Opérations de décalage de bits (shift left/right logical)					
`slli`	d, s, C	001 0011	001	imm[4:0] = C imm[11:5] = 000 0000	$d \leftarrow s \ll C$
`srli`	d, s, C	001 0011	101	imm[4:0] = C imm[11:5] = 000 0000	$d \leftarrow s \gg C$
`srai`	d, s, C	001 0011	101	imm[4:0] = C imm[11:5] = 010 0000	$d \leftarrow s \gg^{*} C$
Opérations de test de conditions (set if less than)					
`slti`	d, s, C	001 0011	010	C codé sur 12 bits	$d \leftarrow 1$ si $(s < C)$ $d \leftarrow 0$ sinon
`sltiu`	d, s, C	001 0011	011	C codé sur 12 bits	$d \leftarrow 1$ si $(s < C)$ $d \leftarrow 0$ sinon
Opérations de transferts ALU (load/add upper immediate [pc])					
`lb`	d, C(s)	000 0011	000	C codé sur 12 bits	$d \leftarrow \mathrm{RAM}[s + C]$
`lbu`	d, C(s)	000 0011	100	C codé sur 12 bits	$d \leftarrow \mathrm{RAM}[s + C]$
`lh`	d, C(s)	000 0011	001	C codé sur 12 bits	$d \leftarrow \mathrm{RAM}[s + C]$
`lhu`	d, C(s)	000 0011	101	C codé sur 12 bits	$d \leftarrow \mathrm{RAM}[s + C]$
`lw`	d, C(s)	000 0011	010	C codé sur 12 bits	$d \leftarrow \mathrm{RAM}[s + C]$
Saut inconditionnel (jump and link register)					
`jalr`	d, s, *Etq*	110 0111	000	*Etq*/C codé sur 12 bits	$d \leftarrow \mathrm{PC} + 4$ $\mathrm{PC} \leftarrow s + C$
Appels divers – (environment call & debuger)					
`ebreak`		111 0011	000	0000 0000 0001	debogueur
`ecall`		111 0011	000	0000 0000 0000	appel système

Format B – Branch

31 … 25	24 … 20	19 … 15	14 … 12	11 … 7	6 … 0
imm[12 | 10:5]	s2	s1	funct3	imm[4:1 | 11]	opcode

Syntaxe Assembleur		**Opcode**	**funct3**	**Immédiat : *Etq***	**Effet**
Saut conditionnel (branch if equal/not equal/less than/greater or equal…)					
`beq`	s1, s2, *Etq*	110 0011	000	Scindé et bi-réparti	$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 = s2$
`bne`	s1, s2, *Etq*	110 0011	001	Scindé et bi-réparti	$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 \neq s2$
`blt`	s1, s2, *Etq*	110 0011	100	Scindé et bi-réparti	$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 < s2$
`bge`	s1, s2, *Etq*	110 0011	101	Scindé et bi-réparti	$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 \geq s2$
`bltu`	s1, s2, *Etq*	110 0011	110	Scindé et bi-réparti	$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 < s2$
`bgeu`	s1, s2, *Etq*	110 0011	111	Scindé et bi-réparti	$\mathrm{PC} \leftarrow \mathrm{PC} + Etq$ si $s1 \geq s2$